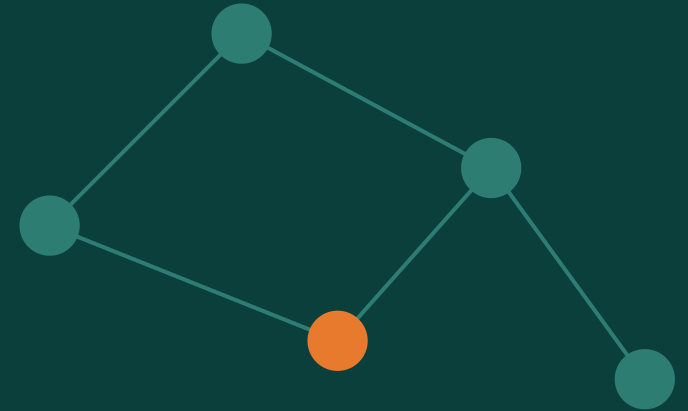




ESTRUTURAS DE DADOS E ALGORITMOS

Algoritmos, Estruturas de Dados e Programas

Aula 1 · as três ideias que sustentam toda a UC

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três ideias, uma equação

1

Algoritmo

O que é, que propriedades tem e como se concebe.

2

Estruturas de Dados

Como organizamos os dados para que os algoritmos trabalhem bem.

3

Programas em Java

Onde as duas ideias se juntam: tipos, arrays e listas.

$$\text{Algoritmos} + \text{Estruturas de Dados} = \text{Programas}$$

**Tem uma lista ordenada com 1 000 000 de nomes.
Quantos passos precisa, no máximo, para encontrar o
seu?**

A $\approx 1\,000\,000$

B $\approx 500\,000$

C $\approx 1\,000$

D ≈ 20

Guarde a sua resposta. Voltamos a ela daqui a pouco.

1

PARTE 1

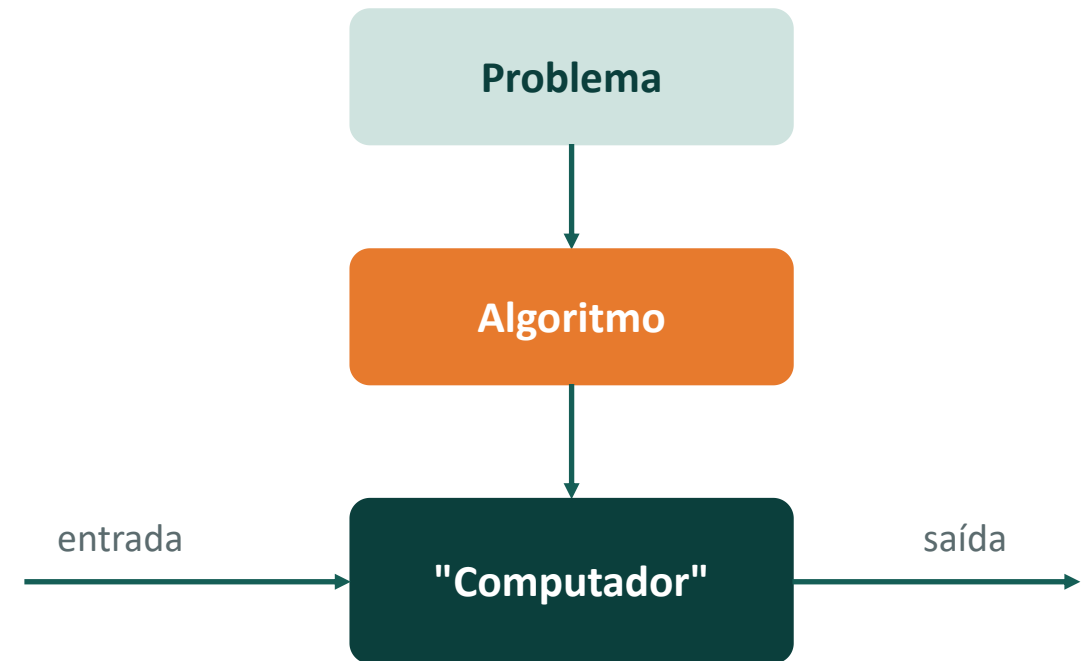
Algoritmo

A receita antes da cozinha.

O que é um algoritmo?

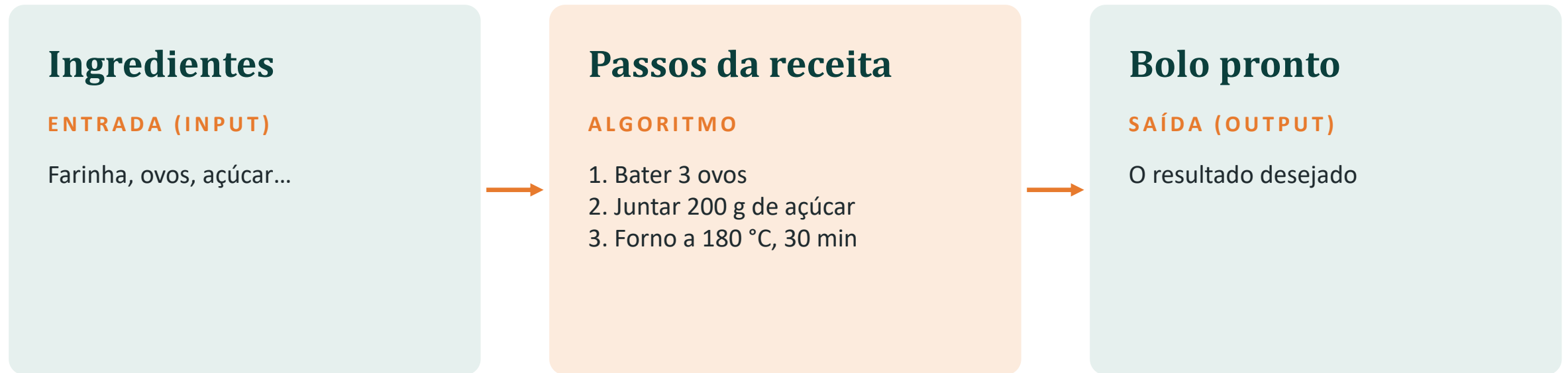
Um algoritmo é uma sequência de **instruções não ambíguas** para resolver um problema, ou seja, para obter um **resultado desejado** para **qualquer entrada legítima** num **tempo finito**.

— adaptado de Levitin (2011)



O "computador" pode ser uma máquina... ou uma pessoa a seguir as instruções.

Um algoritmo é como uma receita



Uma boa receita — e um bom algoritmo — tem de ser:

Não ambígua

Entradas bem definidas

Finita

Correta

Cinco ideias-chave sobre algoritmos

1

Sem ambiguidade

Cada passo tem de ter um único significado. Isto não é negociável.

2

Entradas bem definidas

A gama de entradas para as quais o algoritmo funciona tem de ser especificada com cuidado.

3

Várias representações

O mesmo algoritmo pode ser escrito em português, pseudocódigo, fluxograma ou código.

4

Várias soluções

Podem existir vários algoritmos para resolver o mesmo problema.

5

Velocidades muito diferentes

Ideias diferentes podem resolver o mesmo problema a velocidades dramaticamente diferentes.

As ideias 4 e 5 são a razão de ser desta UC: escolher o algoritmo certo.

Mesmo problema, algoritmos diferentes

Problema: encontrar um nome numa lista ordenada com 1 000 000 de nomes.

Pesquisa sequencial

Ver o 1.º nome, depois o 2.º, depois o 3.º... até encontrar.

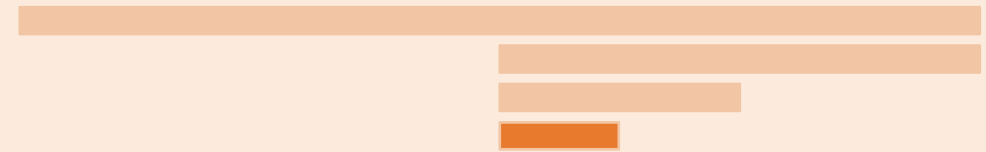


1 000 000

passos no pior caso

Pesquisa binária

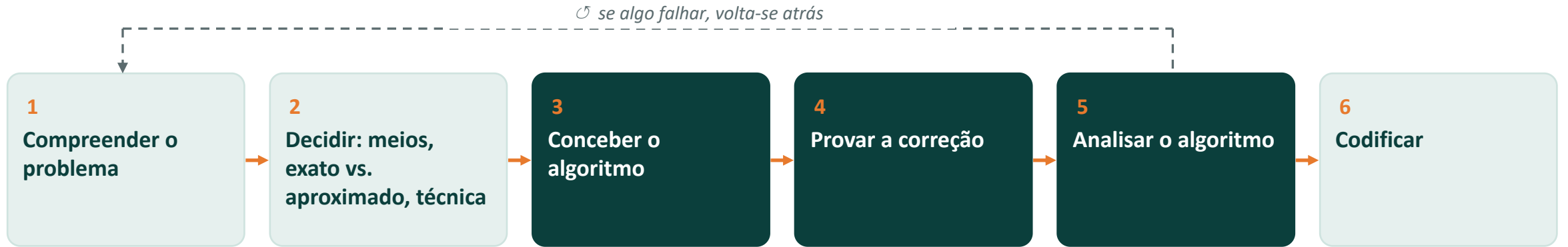
Abrir ao meio; descartar a metade errada; repetir.



≈ 20

passos no pior caso ($2^{20} \approx 1\,000\,000$)

Como se concebe e analisa um algoritmo



3 Conceber

Algoritmos + Estruturas de Dados = Programas. A escolha da estrutura faz parte do desenho.

4 Provar

Funcionar para algumas entradas não prova que o algoritmo está correto.

5 Analisar

Eficiência no tempo e no espaço, simplicidade e capacidade de generalização.

Testei o meu algoritmo com 50 entradas diferentes e deu sempre o resultado certo.

Posso concluir que está correto?

Não.

Os testes podem mostrar a presença de erros, mas não a sua ausência. A correção demonstra-se para todas as entradas legítimas — por exemplo, por indução matemática.

2

PARTE 2

Estruturas de Dados

Arrumar bem para encontrar depressa.

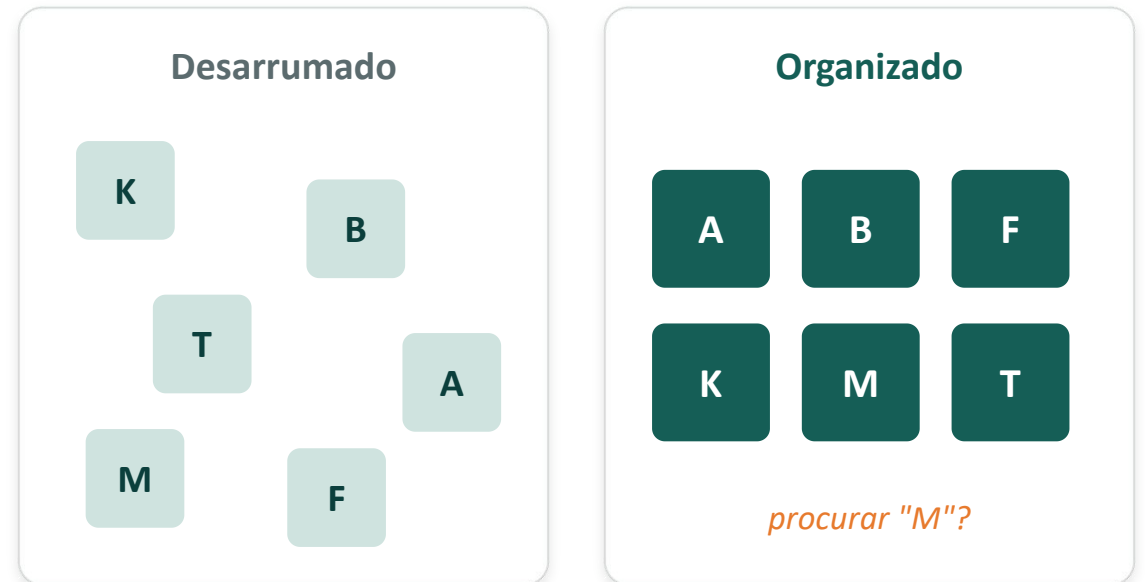
O que é uma estrutura de dados?

Uma estrutura de dados é **um esquema particular de organização de dados relacionados.**

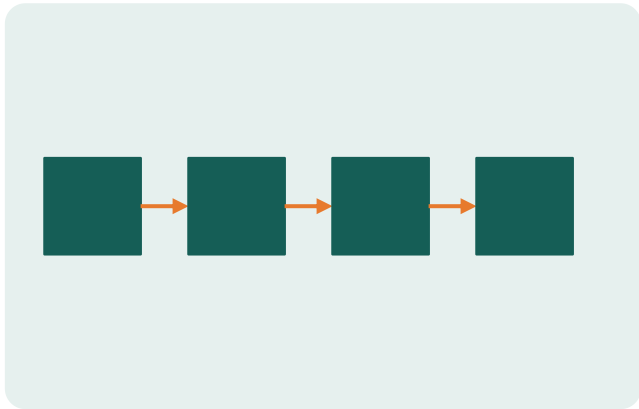
Porque importa?

A grande maioria dos algoritmos opera sobre dados. A forma como os organizamos determina o que o algoritmo consegue fazer — e a que velocidade.

O problema dita a estrutura: desde tipos elementares (inteiros, caracteres) até estruturas complexas (uma matriz = um array de arrays).



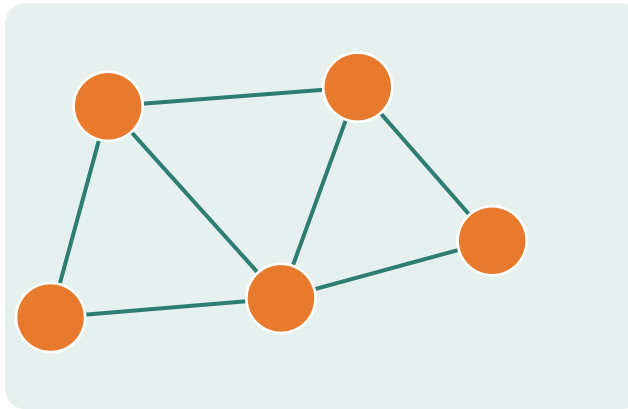
Três famílias fundamentais



Lineares

Elementos em sequência, um a seguir ao outro.

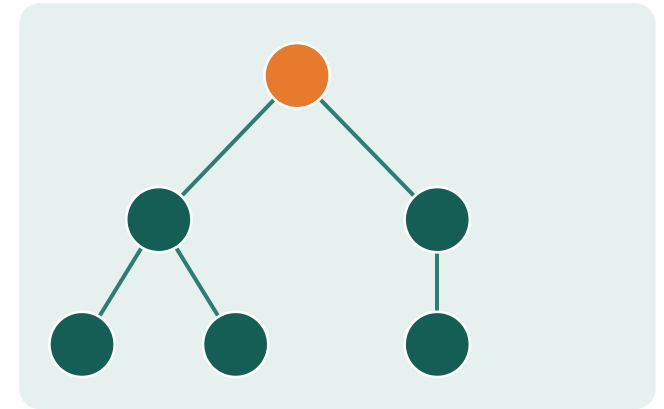
Ex.: array, lista ligada



Grafos

Vértices (nós) ligados por arestas (arcos).

Ex.: redes sociais, mapas



Árvores

Grafos conexos e sem ciclos: há uma hierarquia.

Ex.: pastas, árvore genealógica

3

PARTE 3

Programas

Onde o algoritmo encontra a estrutura.

Algoritmo $\neq$ Programa

Algoritmos

+

Estruturas de Dados

=

Programas

Wirth, N. (1976). Algorithms + Data Structures = Programs.

Algoritmo

- A ideia: o método para resolver o problema
- Independente da linguagem
- Pode ser escrito em português ou pseudocódigo

Programa

- A concretização: implementação específica do algoritmo
- Escrito numa linguagem concreta (ex.: Java)
- Um algoritmo $\rightarrow$ muitos programas possíveis

4

PARTE 4

Estruturas de Dados Básicas em Java

Das ideias ao código.

Os 8 tipos de dados primitivos

Família	Tipo	Tamanho	O que guarda
Inteiros	byte	1 byte	Inteiros de -128 a 127
	short	2 bytes	Inteiros de -32 768 a 32 767
	int	4 bytes	Inteiros de -2 147 483 648 a 2 147 483 647 ($\approx \pm 2,1$ mil milhões)
	long	8 bytes	Inteiros até $\approx \pm 9,2 \times 10^{18}$
Reais	float	4 bytes	Números decimais, ≈ 6 a 7 dígitos de precisão
	double	8 bytes	Números decimais, ≈ 15 dígitos de precisão
Lógico	boolean	1 bit*	true ou false
Carácter	char	2 bytes	Um único carácter Unicode (ex.: 'A', 'ç')

* 1 bit de informação; o espaço efetivamente ocupado depende da JVM. · String não é primitivo: é uma classe.

O mapa do que vamos explorar

ESTRUTURAS ESSENCIAIS

Array

Tamanho fixo, acesso direto por índice

```
int[] v = new int[5];
```

ArrayList

Lista que cresce automaticamente

```
List<String> l = new ArrayList<>();
```

Listas multidimensionais

Listas de listas (tabelas, matrizes)

```
List<List<Integer>> m;
```

LinkedList

Elementos ligados por referências

```
List<String> l = new LinkedList<>();
```

FERRAMENTAS

Interfaces genéricas

List<T>, Collection<T>: programar para o "contrato", não para a implementação.

Iteradores e vistas

Percorrer e observar coleções de forma segura e uniforme.

Array, ArrayList e LinkedList lado a lado

Array



0 1 2 3 4

tamanho fixo

ArrayList



array interno com espaço livre; cresce quando enche

LinkedList



nós ligados por referências

	Array	ArrayList	LinkedList
Tamanho	Fixo	Dinâmico	Dinâmico
Aceder ao elemento i	Imediato	Imediato	Percorre desde o início
Inserir no início/meio	Desloca elementos	Desloca elementos	Só religa referências

Projeto BasicDataStructures

Vamos analisar e discutir o projeto em quatro passos, do mais simples ao mais rico:



Agora é consigo



Aquecimento



Escreva um programa em Java que encontre o valor mínimo e o valor máximo num array de inteiros e calcule a média de todos os valores.



Transferência



Repita o exercício anterior utilizando um ArrayList ou uma LinkedList. O que muda no código? O que fica igual?



Desafio



Implemente a sua própria versão da classe ArrayList para Strings, com os métodos add, get e size. O array interno começa com 10 posições e cresce 5 posições sempre que fica cheio.

O que acontece quando o array enche?

size = 7, capacidade 10



size = 10 → cheio!



add(): novo array de 15



Algoritmo do add(x):

1. se `size == capacidade` → criar array com capacidade + 5 e copiar os elementos
2. guardar `x` na posição `size`
3. incrementar `size`

Três ideias a reter

1

Algoritmo

Sequência finita de instruções não ambíguas que resolve um problema para qualquer entrada legítima.

2

Estrutura de dados

Forma de organizar dados relacionados. A escolha certa torna o algoritmo mais simples e mais rápido.

3

Programa

Implementação concreta de algoritmos + estruturas de dados numa linguagem — em Java, nesta UC.

Próxima aula: como medir a eficiência de um algoritmo.

Bibliografia

1

Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms* (3.^a ed.). Pearson.

2

Wirth, N. (1976). *Algorithms + Data Structures = Programs*. Prentice-Hall.

3

W3Schools. *Java Data Types*. https://www.w3schools.com/java/java_data_types.asp

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt